A smaller raw clausal presentation of intuitionistic primitive recursive arithmetic

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give a further reduction of the current *raw clausal* proposal in ProveIt's ConciseIPRA collection. Terms remain raw scalar iterator expressions; formulas are equations and one binary binding constructor D. Contexts contain reusable clauses. There are five inference patterns: reflexivity, computational equality, clause use, paired elimination, and paired introduction. Computational equality has two cases, instead of the three in the existing clausal proposal. The removed case is simulated by two zero-iteration steps through a freshly constructed frame, with no induction or normalization. Consequently every sequent is preserved, and a proof with N inference nodes and k removed-case nodes translates to one with exactly $N + 2k$ nodes. We spell out the theory, the reconstruction of equality and logic, and both interpretation composites with full first-order iPRA. The archive contains complete Python checkers, a proof translator, reproducible measurements, and adversarial regression checks. This is a verified composition of two reductions already present separately in the repository, not a claim of an absolute minimum or a new arithmetic-strength theorem.

1 Result and comparison point

The reference is the ConciseIPRA directory at commit

`5d85023b0688e03b75f0ec3a04bb43ef0bb51705`.

It was inspected through GitHub on November 11, 2026, Pacific time [1]. Its adopted six-pattern calculus already has two-case computational equality. Its `raw_clausal` package removes the separate Q constructor and hypothesis pattern, but retains a redundant third case of evaluation. The proposal here combines these two reductions [2, 3]. The comparison is to that exact snapshot, not to an earlier eight-rule kernel. The relevant source paths are listed in Section 11.

Let $\mathcal{C}_3$ denote the existing raw clausal calculus and $\mathcal{C}_2$ the calculus below. The principal statement is

$$\Gamma \vdash_{\mathcal{C}_3} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_2} F$$

for every finite retained clause context Γ and every native formula F. Terms, formulas, contexts, witnesses, and ordinary capture-avoiding substitution are unchanged. One family of primitive evaluation instances is deleted. In particular, this reduction does not transfer computation into a normalizer or into an external proof search procedure.

The checker comparison has two layers. A literal deletion in the upstream implementation gives a tightly controlled measure of the mathematical change. A second, readable implementation shares its

syntax traversal and infers the conclusion of a paired elimination; it has its own matched three-case control. These implementation changes are identified separately. The collection's historical compact encodings are smaller than these readable modules, so we do not claim a repository-wide byte record. There is also no language-independent theorem that five displayed patterns is the fewest possible.

2 Complete specification of the theory

2.1 Numerical terms, formulas, and clauses

There is one numerical sort. Terms are finite raw expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound. The counter n and initializer a lie outside that binder. Other numerical variables in h are passive parameters. An iterator is a term constructor with a bound state variable, not quantification over functions. There are no additional function symbols or universal program evaluator.

Formulas are

$$F ::= (s = t) \mid \mathsf{D}_{x,y}F.$$

Both displayed coordinates of D are bound. We work modulo renaming bound variables and use capture-avoiding substitution. All other syntactic equality tests below are literal tests on these raw expressions. In particular, $I(0, a; u.h)$ is not identified definitionally with a.

A context is a finite list of reusable clause occurrences

$$[A;\, y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

The clause binds y only in its consequent. Clauses are context objects, not a third formula constructor. Put $H = (0 = 0)$ and write

$$\mathrm{ord}(F) := [H;\, v.F], \qquad v \notin \mathrm{FV}(F).$$

Thus Γ, F in a rule abbreviates $\Gamma, \mathrm{ord}(F)$. Occurrences are retained separately even if their clause values are identical. The designated contradiction is

$$\bot_* := (0 = S0).$$

2.2 Two-case computational equality

For terms s, t, b, the partial operation $\Phi_2(s, t, b)$ is defined by exactly the following table:

Shape of b	Required source s	Result $\Phi_2(s, t, b)$
$I(0, a; u.h)$	a	t
$I(Sn, a; u.h)$	$I(n, a; u.h)$	$h[t/u]$

Every required equality in this table is a literal syntax comparison. The operation is undefined in every other case. In particular, $b \equiv s$ is no longer an evaluation case.

2.3 Every primitive axiom and inference rule

The entire theory consists of the following five patterns:

$$\frac{}{\Gamma \vdash s = s} \qquad \text{Refl,} \tag{1}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi_2(s,t,b)} \qquad \text{Eval, if defined,} \tag{2}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \qquad \text{Use, if } [A; y.B] \text{ occurs in } \Gamma, \tag{3}$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, [B(b,0); y.B(b,y)] \vdash F}{\Gamma \vdash F} \qquad \text{Elim,} \tag{4}$$

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)} \qquad \text{Intro,} \tag{5}$$

where

$$\eta = \begin{cases} z, & B \text{ is literally an equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

In Elim, b is fresh for Γ, F, and all passive parameters of the matrix. In Intro, z is fresh for Γ, t, and all passive matrix parameters. Supplied syntax is well scoped. Witnesses and specialization targets may be arbitrary numerical terms.

Refl is the only axiom schema. There are no further primitive logical, equality, computation, successor, or induction axioms. The two rows of Eval are part of the specification, as is the mode test in Intro. Counting them explicitly avoids disguising a case distinction as a single unexplained rule. The calculus has five displayed patterns, with two evaluation cases and two introduction modes.

2.4 Structural operations

Ordinary hypothesis use is derived: prove H by Refl at 0 and apply Use to $[H; v.F]$. Weakening and injective fresh renaming follow by induction on a derivation. Ordinary-assumption cut is also admissible. For a distinguished occurrence $[H; v.F]$, replace each Use selecting that occurrence by the supplied proof of F, weakened to the current context. Freshen eigenvariables before doing so. All other nodes are transformed recursively.

These elementary transformations do not change numerical witnesses. We do not assume that arbitrary substitution into a derivation preserves every rule tag: a nonzero raw witness can become literally 0, changing Intro's mode. Full substitution admissibility for ordinary formula sequents will follow from the equivalence proof.

3 The deleted equality case

The extra case in $\mathcal{C}_3$ is

$$\frac{s = t \qquad s = r}{r = t}. \tag{Root}$$

This is ordinary equality composition with a shared left side. The following identity-frame argument is the same reduction already proved for the six-pattern calculus in `equality_from_iteration` and `NativeEval.lean`; here it is applied inside the clausal calculus [3, 4].

Lemma 3.1 (Root evaluation is derived)**.** *Given $p : \Gamma \vdash s = t$ and $q : \Gamma \vdash s = r$, two zero evaluations and one reflexivity leaf derive $\Gamma \vdash r = t$. Neither premise is duplicated.*

Proof. Let $J_s = I(0, s; u.0)$. Refl gives $J_s = J_s$. Zero evaluation with the major premise $q : s = r$ gives

$$J_s = r.$$

Zero evaluation of this result with major premise $p : s = t$ gives $r = t$. In proof notation,

$$\mathrm{Root}(p, q) := \mathrm{ev}_0(p, \mathrm{ev}_0(q, \mathrm{refl}(J_s))).$$

The initializer of J_s is literally s in both steps, so both source checks succeed. The update body is unused and may equally well be the bound identity variable. No numerical evaluation of s is performed. □

Theorem 3.2 (Exact equivalence with the current clausal proposal)**.** *For every retained clause context Γ and formula F,*

$$\Gamma \vdash_{\mathcal{C}_3} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_2} F.$$

The translations preserve every term, formula, and retained assumption of the end sequent.

Proof. Every primitive $\mathcal{C}_2$ inference is already a $\mathcal{C}_3$ inference. In the other direction, recurse over a finite $\mathcal{C}_3$ proof and replace each Root occurrence by Lemma 3.1. All other inferences, including their freshness conditions and Intro modes, are unchanged. The construction works under arbitrary clause contexts, including those opened inside Elim and Intro. □

Proposition 3.3 (Exact inference-node cost)**.** *If the input proof has N nodes and k Root nodes, the translation has exactly $N + 2k$ nodes, hence at most $3N$ nodes.*

Proof. Each removed node becomes two Eval nodes and one Refl node. Its two original premise trees occur once each. Every other node is copied once. Sum these local changes over the input tree. □

This statement counts inference nodes. It is not a bound on serialized JSON bytes or running time: term labels are copied into the new frame.

4 Equality and arithmetic computation are recovered

Write R_s for Refl at s. Lemma 3.1 immediately gives

$$\mathrm{sym}(p : s = t) = \mathrm{Root}(R_s, p) : t = s,$$

and, for $q : t = v$,

$$\mathrm{trans}(p, q) = \mathrm{Root}(q, \mathrm{sym}(p)) : s = v.$$

Both iterator equations are derived, not additional axioms:

$$\mathrm{ev}_0(R_a, R_{I(0,a;u.h)}) : I(0, a; u.h) = a,$$
$$\mathrm{ev}_S(R_{I(n,a;u.h)}, R_{I(Sn,a;u.h)}) : I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

Lemma 4.1 (Congruence without a primitive template rule)**.** *For every fixed numerical term template $h(u)$,*

$$s = t \quad \Longrightarrow \quad h(s) = h(t).$$

This includes templates with several occurrences of u and occurrences inside nested iterator bodies.

Proof. Put $A_s = I(0, s; u.h)$ and $B_s = I(S0, s; u.h)$. For $p : s = t$ form

$$\begin{aligned} z_s &= \mathrm{ev}_0(R_s, R_{A_s}) && : A_s = s, \\ c_s &= \mathrm{ev}_S(z_s, R_{B_s}) && : B_s = h(s), \\ z_{s,t} &= \mathrm{ev}_0(p, R_{A_s}) && : A_s = t, \\ \mathrm{ev}_S(z_{s,t}, c_s) & && : h(s) = h(t). \end{aligned}$$

The successor source is literally A_s in each use. Capture-avoiding body substitution supplies the appropriate free occurrences. The input equality is used once. □

Corollary 4.2 (Crossed transport and equation transport)**.** *The following rules are admissible:*

$$s = t,\ h(s) = r \quad \Longrightarrow \quad r = h(t),$$

$$s = t,\ a(s) = b(s) \quad \Longrightarrow \quad a(t) = b(t).$$

Proof. Apply Root to $h(s) = h(t)$ and $h(s) = r$. For the second claim, apply the first rule with templates a and b successively: $b(s) = a(t)$, then $a(t) = b(t)$. □

Lemma 4.3 (Case introduction for every witness)**.** *For every matrix B, every witness t, and fresh z,*

$$\Gamma, B(t, 0) \vdash B(t, Sz) \quad \Longrightarrow \quad \Gamma \vdash \mathsf{D}_{x,y} B(x, y).$$

Proof. Only an equation matrix at literal witness 0 is missing from the primitive case mode. Put $c = I(0, 0; u.0)$. This term is not literally 0, and derived zero computation proves $c = 0$. Under $B(c, 0)$, equation transport gives $B(0, 0)$. Cut in the supplied branch to obtain $B(0, Sz)$ and transport back to $B(c, Sz)$. Primitive Intro at witness c now uses its case mode and concludes the original paired formula. Only equation transport was used; transport through a compound formula is not assumed. □

Lemma 4.4 (Transport in every native formula)**.** *For any Eq/D template $F(v)$, $s = t, F(s) \vdash F(t)$ is admissible.*

Proof. Use external structural induction on F. The equation case was proved above. For $F(v) = \mathsf{D}_{x,y} B(v, x, y)$, eliminate $F(s)$ at fresh b, obtaining $[B(s, b, 0); y.B(s, b, y)]$. Introduce $F(t)$ by Lemma 4.3, with witness b. Under its local assumption $B(t, b, 0)$, use the induction hypothesis backwards to obtain $B(s, b, 0)$, Use at Sz to obtain $B(s, b, Sz)$, then the induction hypothesis forwards to obtain $B(t, b, Sz)$. Close the introduction and elimination. Recursive transports concern strictly smaller formulas. □

5 Conditional closure and ordinary intuitionistic logic

5.1 Closure as a macro

Define, with w unused,

$$q_y B(y) := \mathsf{D}_{w,y} B(y).$$

Case introduction proves $q_y B$ from $B(0) \vdash B(Sz)$. Elim on $q_y B$ opens exactly $[B(0); y.B(y)]$, independently of the new witness; Use then gives

$$q_y B,\ B(0) \vdash B(t).$$

Conversely a retained clause $[B(0); y.B(y)]$ proves $q_y B$: assume its base, specialize it at Sz, and use case introduction.

For a literal equation $E(y)$, Intro at unused witness 0 gives $q_y E$ from $E(z) \vdash E(Sz)$. Hence

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \qquad \text{(Equation induction)}$$

is derived. For arbitrary F, case introduction and specialization similarly give the numerical case rule

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}, \qquad \text{(Numerical cases)}$$

where z is fresh for retained parameters. The distinction matters: unrestricted progress induction for compound formulas is not a primitive rule.

5.2 Explicit logical macros

We record the entire interface so that no ordinary logical rules are silently added to the kernel. Write $1 = S0$, and define

$$C(v, a, b) = I(v, a; u.b), \quad \delta(v) = C(v, 1, 0), \quad p(v) = I(v, 0; u.\delta(u)).$$

The update variable in C is unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$.

The gate of a formula is defined recursively by

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} G_v B.$$

Bound variables are freshened away from v. It satisfies $\vdash G_0 A$ and $G_{Sz} A \dashv\vdash A$. An equation can be padded by $E \dashv\vdash \mathsf{D}_{x,y} G_y E$ for fresh unused x, y.

A selector between arbitrary formulas is defined recursively by

$$\begin{aligned} M_v(s = t, a = b) &= (C(v, s, a) = C(v, t, b)), \\ M_v(\mathsf{D}_{x,y} A, \mathsf{D}_{x,y} B) &= \mathsf{D}_{x,y} M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y} B) &= \mathsf{D}_{x,y} M_v(G_y E, B), \\ M_v(\mathsf{D}_{x,y} A, E) &= \mathsf{D}_{x,y} M_v(A, G_y E), \end{aligned}$$

where E is an equation and the binding coordinates are aligned and fresh. Each recursive call decreases the maximum binder depth. Gate padding and binder congruence give $M_0(A, B) \dashv\vdash A$ and $M_{Sz}(A, B) \dashv\vdash B$.

The ordinary connectives are the following macros, with all new coordinates fresh and w unused:

$$\begin{aligned} A \to_* B &:= q_z M_z(A, B), \\ A \wedge_* B &:= q_z G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* v\, M_v(A, B), \\ \forall_* y\, A(y) &:= A(0) \wedge_* q_y A(y), \\ \bot_* &:= (0 = S0). \end{aligned}$$

Their introduction and elimination rules are derived as follows. Implication uses the selector's zero and successor endpoints. Existential introduction uses the given numerical witness and the successor gate; existential elimination opens a fresh witness, proves the zero gate, and specializes at 1. Disjunction uses witnesses 0 and 1; its elimination uses numerical cases on the selector control after existential elimination. Conjunction uses a true zero gate, while the specializations $z = 1, 2$ recover its two operands, since $p(1) = 1$ and $p(2) = 0$. For its introduction, from A and B use numerical cases at an arbitrary control v to derive $M_v(A, B)$; instantiate $v = p(Sz)$, apply the successor gate, and introduce the closure. Universal introduction packages a zero instance and a closure; elimination uses the closure and its base.

From $0 = 1$, equation transport and selector computation imply $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary terms. External induction on formula syntax, using case introduction at the paired step, then derives every formula. This proves falsity elimination. These constructions recover intuitionistic logic; excluded middle for arbitrary formulas is not postulated.

6 Exact link to the formalized native calculus

The repository's $\mathcal{E}_2$ (Lean name `EPrf`) uses the same raw numerical terms and Eq/Q/D formulas. Its six patterns are Hyp, Refl, two-case Eval, QE, DE, and the same witness-directed introduction. Its equivalence to the native kernel is the theorem `eprf_iff_nprf` in `NativeEval.lean` [4]. We use this established bridge explicitly; the new checker has not been verified by Lean.

Define τ from Eq/Q/D formulas to Eq/D formulas by

$$\tau(s = t) = (s = t), \qquad \tau(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} \tau(B), \qquad \tau(\mathsf{Q}_y B) = \mathsf{D}_{w,y} \tau(B),$$

where w is fresh and unused. Let j be inclusion of the Eq/D language. Numerical terms are unchanged.

Theorem 6.1 (Faithfulness in both directions)**.** *For every native context Γ and formula A,*

$$\Gamma \vdash_{\mathcal{E}_2} A \quad \Longrightarrow \quad \mathrm{ord}(\tau\Gamma) \vdash_{\mathcal{C}_2} \tau A.$$

For every Eq/D context Δ of ordinary formula assumptions,

$$\mathrm{ord}(\Delta) \vdash_{\mathcal{C}_2} F \quad \Longleftrightarrow \quad j\Delta \vdash_{\mathcal{E}_2} jF.$$

Proof. For the forward direction, translate a proof by induction. Hyp is ordinary-assumption use, Refl and Eval are unchanged, and QE is the derived closure specialization. Introduction is copied: τ preserves equation versus compound roots and leaves witnesses alone, so it preserves the selected

mode. At DE, the new Elim opens a clause $[\tau B(b,0); y.\tau B(b,y)]$. That clause derives $q_y\tau B(b,y)$ by the closure interface. Cut this proof into the translated minor branch, whose old assumption was $\tau(\mathsf{Q}_y B(b,y))$, and close Elim.

For the reverse direction, retain the provenance of each clause occurrence. An initial ordinary clause and a local Intro clause $[H; v.G]$ are represented by the native assumption jG. A clause introduced by Elim is represented by $\mathsf{Q}_y jB(b,y)$. At Use, an ordinary clause translates to Hyp and an Elim clause translates to QE, using its translated antecedent proof. Refl and Eval are unchanged. Elim becomes DE, and Intro has the same matrix, witness, and local assumption. Provenance belongs to occurrences, so equal-looking clauses with different origins cause no ambiguity. This proves the reverse implication. The forward implication on the common language follows because τj is the identity. □

Theorem 6.2 (Both formula composites)**.** *For every Eq/*D *formula* F, $\tau jF = F$ *up to bound renaming. For every native formula* A,

$$A \dashv\vdash_{\mathcal{E}_2} j\tau A.$$

Both numerical translations are identities.

Proof. The essential equivalence is $\mathsf{Q}_y B \dashv\vdash \mathsf{D}_{w,y} B$ with w unused. Forward, assume $B(0)$, specialize the closure at Sz, and introduce the paired formula by cases. Backward, DE opens precisely $\mathsf{Q}_y B$ and Hyp closes the branch. Uniform pointwise interderivability lifts through D: eliminate the source, introduce the target by cases at the opened witness, convert its base backwards, specialize the source clause, and convert the result forwards. The same argument works for Q. External induction on A proves the claim. □

The argument proves more than agreement on closed equations or on standard natural numbers. It preserves every ordinary retained-context sequent and supplies both formula round trips. Arbitrary external clauses can also be represented through the derived formula $A \to_* \forall_* yB$, using the logical interface above.

7 Bi-interpretability with full first-order iPRA

7.1 The precise target and ordinary realization

The target iPRA is intuitionistic first-order logic with equality, one function symbol for each fixed primitive-recursive definition, its defining equations, successor injectivity, $Sx = 0 \to \bot$, and contextual induction for quantifier-free formulas, with numerical parameters. This is the repository's `PRAAx` theory.

For an ordinary first-order reading, elaborate each fixed iterator body into a function symbol with the two iterator equations. Let T_{eq} use ordinary intuitionistic equality logic, these equations, contextual literal-equation induction, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad 0 = S0 \to \bot.$$

The native reading is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x(B^\circ(x,0) \to \forall y\, B^\circ(x,y)),$$

and a clause reads as $A^\circ \to \forall y B^\circ$. The explicit separation sentence must not be omitted from this ordinary realization. Positive equations alone admit the singleton structure; defining native falsity as $0 = 1$ is not a reason to drop the ordinary falsity bridge.

7.2 Arithmetic translation, in the required dependency order

The repository has a substantial internal proof of $T_{\mathrm{eq}} \equiv$ iPRA, not merely a standard-model interpretation. The five supporting mathematical source files are included in `reference/foundations/` [5]. They establish, in order:

1. A scalar predecessor P with $P0 = 0$ and $P(Sn) = n$, using equation induction and numerical cases, before pairing or quantifier-free induction is available.
2. Truncated subtraction and a term reflecting actual numerical equality throughout the whole domain.
3. A characteristic term for each fixed quantifier-free formula, giving every quantifier-free induction instance from equation induction.
4. Pairing with projections and a compiler for every fixed primitive-recursive definition.

This dependency order prevents a circular appeal to predecessor, pairing, or a Boolean equality detector during their own construction. The inherited details are not claimed as new results of this note.

For clarity, the final recursion compiler is explicit. If

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

and π, L, R are proved pairing and projection terms, set

$$\begin{aligned} V(n, \vec{p}) &= I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), R(u), \vec{p}))\big), \\ f^*(n, \vec{p}) &= R(V(n, \vec{p})). \end{aligned}$$

Equation induction proves $L(V(n, \vec{p})) = n$; projection equations then establish both defining equations of f^*. The construction recurses externally over each fixed finite function description. No variable ranges over arbitrary descriptions and no uniform total evaluator is asserted inside iPRA.

In the other direction, every fixed iterator is primitive recursive: its update ignores the recursion-counter argument. Thus each scalar term has a direct iPRA interpretation.

7.3 Why mutual interpretability is not the whole claim

Theorem 7.1 (Whole-domain bi-interpretation and equiconsistency)**.** *The ordinary realization presented by $\mathcal{C}_2$ is bi-interpretable with full first-order iPRA. Both interpretations use the whole numerical domain and actual equality. The graph of both numerical composite isomorphisms is $x = y$. The theories are equiconsistent.*

Proof. Theorem 6.1 and the formula-composite theorem give inverse interpretations between $\mathcal{C}_2$ and the formalized raw native calculus, with the identity on numerical terms. The repository proves the native/T_{eq} equivalence and the T_{eq}/iPRA bi-interpretation. We compose these established interpretations.

The internal reason for the numerical composites is uniqueness. Two fixed iterators with the same initializer and provably identical updates are equal at all counters by equation induction on their value equality. The same argument holds for each ordinary primitive recursion. Induction on a finite function description shows that its double translation satisfies its original defining equations and is therefore provably equal to the original operation. These are identities at arbitrary internal arguments, not statements just about standard numerals. Domain restriction or quotient equality is never introduced, so $x = y$ is the definable composite isomorphism.

The logical macros have their ordinary meanings. In particular,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \to_* \forall_* y B(x,y)\big).$$

Forward, Elim opens a clause; under its base it supplies all instances, allowing universal, implication, and existential introduction. Backward, eliminate the existential witness and introduce the paired formula by cases, using implication and universal elimination in the successor branch. Structural induction supplies all formula composites.

Finally, $0 = S0$ corresponds to ordinary falsity in both directions. Finite proof translations therefore take a contradiction in either theory to a contradiction in the other, proving equiconsistency. No claim that either theory proves its own consistency is made. □

8 Complete checker and trusted boundary

The delivered `code/checker.py` is a standalone module. It reads one JSON value, verifies the declared closed goal, and prints `ACCEPT` on success. Exceptions produce `REJECT` and a nonzero exit code. It imports only Python's `json` and `sys` modules. No theorem library, normalizer, solver, or external proof checker is called.

8.1 Input and syntax

The top-level object has exactly the fields

```
{"goal": F, "proof": P}
```

The public interface accepts closed theorems. Open hypotheses and clauses appear only in locally checked branches. A sequent with ordinary assumptions can also be internalized using the derived implication and universal macros.

JSON expression	Meaning
`['z']`	0
`['v',i]`	numerical variable
`['s',t]`	$S(t)$
`['i',n,a,h]`	$I(n, a; u.h)$, one binder in h
`['=',s,t]`	equation
`['d',B]`	paired formula, two binders in B

The table uses Python quoting for readability; certificates use JSON double quotes. Positive variable indices are locally bound indices, starting at 1. Negative integers name fresh free levels opened by

the checker. At scope k and local binder depth d, exactly

$$-k \le i \le d, \qquad i \neq 0$$

is allowed. Booleans and floating-point values are not integer indices. The root scope is zero. In a paired matrix, index 1 is y and index 2 is x. A fresh branch parameter is the new negative level $-k-1$.

One recursive traversal implements constructor typing, binder depth, and bound-variable opening. In an opening of a binder beyond depth d, index $d+1$ is replaced, larger indices are decreased by one, and all others are unchanged. Opened values have no ambient positive indices: they are well-scoped terms at depth zero. Consequently they can be inserted beneath local binders without capture; their negative free levels remain distinct from all bound indices.

8.2 Proof packets

Packet	Rule
`['r',t]`	Refl at t
`['v',p,q]`	Eval on two derived equations
`['u',j,t,p]`	Use clause occurrence j at target t
`['e',p,q]`	Elim; paired major p, local branch q
`['i',B,t,p]`	Intro of $\mathsf{D}B$, witness t, branch p

The elimination packet does not repeat its intended conclusion. Instead, the checker infers the branch conclusion and validates it again in the parent scope. This excludes an escaping eigenparameter. It is an implementation simplification, not a new logical rule.

8.3 Acceptance invariant

Proposition 8.1 (Soundness of the checker algorithm)**.** *If the checker returns a formula at scope k under a well-scoped retained clause context, that formula is well scoped at k and has a derivation in $\mathcal{C}_2$ from that context. Conversely, every finite closed $\mathcal{C}_2$ derivation has a certificate in this format.*

Proof. Induct on a successful proof-tree computation. Refl validates its term. Use checks a nonnegative in-range integer occurrence index, validates its target term, and compares the derived premise to the retained antecedent. Opening the consequent is then the formal Use instance. At Eval, successful derivations yield formulas of exactly the declared grammar. Destructuring three fields therefore forces an equation; destructuring four fields for its left term forces an iterator. The counter check and source equality select precisely one of the two permitted evaluation rows. The returned equation is its formal conclusion.

Elim checks that the major is paired, opens its outer witness at the new negative level, and adds the corresponding clause. The recursive branch is well scoped at $k+1$. Validating its result at k supplies the eigenvariable condition, so Elim is sound. Intro validates its matrix and witness before opening them, constructs the prescribed local assumption using exactly the stated mode test, and compares the recursively derived branch with the required successor instance. Its witness cannot mention the new level.

For completeness, encode a finite derivation after fresh renaming. Use the corresponding packet at each node; choose retained occurrence indices and encode each eigenparameter by the next negative

level. The same induction verifies each packet. This is an idealized unbounded finite-tree algorithm statement. An actual Python process can reject an otherwise valid extremely deep certificate through its finite recursion or memory limits; it does not accept an invalid certificate as a result. □

The shared walker preserves local binder depths for iterators and paired matrices separately. It neither evaluates terms nor compares their numerical denotations. In particular, equality source matching is syntactic, and $0 = 1$ is never accepted because two Python values happen to compare equal after an unintended coercion.

9 Measurements and executable verification

The metric scripts use CPython 3.12.14. A module's leading docstring is removed, then `ast.unparse` is applied and one final newline is added. Comments and formatting are therefore not counted in the normalized source metric. Raw UTF-8 size is recorded separately. Significant tokens exclude layout, comment, encoding, and end-marker tokens. AST nodes count `ast.walk` on the normalized source. Gzip uses level 9 and timestamp zero; it is reported for information. No compressed or dynamically decoded checker is used as the kernel.

Complete checker	Bytes	Tokens	AST	Gzip-9
Pinned three-case clausal checker	3046	853	993	990
Same source, root case deleted	2999	848	986	979
New matched three-case control	2861	808	888	985
New two-case checker	2787	794	868	972

The first pair isolates the mathematical deletion in the upstream certificate format. The second pair isolates the same deletion in the new implementation. Comparing the two pairs also includes a different syntax encoding, a shared traversal, and inferred elimination conclusions; those savings must not all be credited to a new axiom reduction. The compact historical family is reported only as context and is not a matched control for this change.

The direct-deletion suite passed **47 translated closed proofs**, including 32 inherited and 15 added examples; **28 rejection cases**; five obsolete certificates that require translation; and six open-context checks. It executed **320 public CLI runs** across readable and normalized modules, both normally and with `python -O`. It removed 61 root-evaluation nodes and checked the exact node identity at every transformed subtree. The independently implemented readable checker passed **32 positive and 29 rejection examples**, **122 public CLI runs** with ordinary and optimized Python, and **1,000 deterministic substitution comparisons**. These suites overlap; their case counts are not presented as a count of distinct mathematical theorems.

The verification artifacts contain the exact results and reproduction commands. Checks exercise computation, equality composition, multi-occurrence and nested-body congruence, clause specialization, both introduction modes, compound paired formulas, and freshness. Rejection cases cover malformed syntax, source mismatches, wrong branch targets, invalid clause indices, and eigenvariable escape. The translator replaces the omitted root row and compares conclusions under both checkers. It also checks the exact inference-node formula.

These are executable regression and adversarial checks. They are not a proof-assistant verification of Python, the new clause calculus, or the entire bi-interpretation. The universal correctness arguments are the mathematical proofs above; the inherited Lean results are named and pinned separately.

10 Limits and next deletions

10.1 Why reflexivity has not simply been removed

The current raw-clausal report gives a useful obstruction [6]. Fix any finite collection of true closed equation seeds and let A be their finite set of ordinary numerical values. Interpret an atomic equation $s = t$ as

$$[\![s]\!] = [\![t]\!] \in A,$$

and interpret the paired formulas and clauses by their ordinary conditional-universal readings on standard natural numbers. The remaining rules preserve this interpretation: an evaluation's minor equation forces its output's left value into A, and ordinary computation equates the output values; the logical rules use ordinary cases and induction on standard numbers. Thus these finitely many seeds cannot yield $n = n$ for a numeral outside A.

Consequently replacing Refl by $0 = 0$, or by any finite collection of true closed seeds, fails with the other rules fixed. Adding the permanent clause $[0 = 0; y.y = y]$ restores reflexivity, but that merely relocates its universal content into initialization.

10.2 Neither raw evaluation row can be dropped unchanged

If the zero row is deleted, interpret $I(n, a; u.h) = h^n(Sa)$ on standard natural numbers, with actual equality. Successor evaluation, Refl, and all the logical rules remain sound, but $I(0, 0; u.0) = 0$ is false. If the successor row is deleted, interpret $I(n, a; u.h) = a$ for every counter. Zero evaluation and the other rules are sound, but successor computation fails. These are relative irredundancy results for this fixed language and rule set; they do not rule out a different arithmetic encoding.

10.3 What has and has not been achieved

The result is a strict deletion of primitive evaluation instances from the current five-pattern raw clausal theory, with unchanged consequences and a smaller complete checker in controlled comparisons. It also combines the repository's two-case equality interface with its smaller Eq/D formula grammar. It is an incremental composition of existing ideas, supported by a full equivalence argument and a checked proof translation. It does not establish absolute minimality, improve the historical compact byte record, or replace the repository's formalized arithmetic bridge with a new proof.

11 Sources and reproducibility

All repository references below are to commit `5d85023b0688`. Exact paths and source hashes appear in `evidence/source_manifest.json`. Included source copies preserve the upstream license; they are separated from new code and exposition.

References

[1] ProveIt Contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, ConciseIPRA README at the pinned commit.

[2] ProveIt Contributors, *Raw clausal iPRA with computational equality*, `Research/raw_clausal/theory/equivalence.tex` and `code/clausal_eval.py`. Pinned package.

[3] ProveIt Contributors, *Equality from iteration*, `Research/equality_from_iteration/report.tex`. Pinned report.

[4] ProveIt Contributors, `Lean/ConciseIPRA/NativeEval.lean`, especially `EPrf.evalId`, `eprf_iff_nprf`; `NativeBiInterpretation.lean`, especially `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, `native_inconsistent_iff`; and `BiInterpretation.lean`, especially `biInterp`. Pinned Lean sources.

[5] ProveIt Contributors, `Research/certified_zero/reference/`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, `primitive_recursion.tex`. Pinned mathematical bridge.

[6] ProveIt Contributors, *A reflexivity obstruction*, `Research/raw_clausal/theory/reflexivity_obstruction.tex`. Pinned obstruction proof.

A The delivered kernel

The following is the full trusted Python module, apart from the Python runtime and standard JSON parser. The translator, test generator, size measurement scripts, and report build are not trusted by this checker.

```
"""Strict raw clausal iPRA checker with two-case computational equality.

Terms are ['z'], ['v', i], ['s', t], and ['i', n, a, h].
Formulas are ['=', s, t] and ['d', B]. Positive variables are bound
indices; negative variables are fresh free levels. Input is {'goal': F,
'proof': P}. Clause contexts are maintained by the checker.
"""
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('invalid certificate')


def walk(a, variable, depth=0, sort=-1):
    require(type(a) is list and a)
    tag, *args = a
    require(sort < 0 or (tag in ('=', 'd')) == sort)
    if tag == 'v':
        require(len(args) == 1)
        return variable(args[0], depth)
    binders = {'z': [], 's': [0], 'i': [0, 0, 1], '=': [0, 0], 'd': [2]}[tag]
    require(len(args) == len(binders))
    return [tag] + [walk(t, variable, depth + k, tag == 'd')
                    for t, k in zip(args, binders)]


def syntax(a, scope, sort=0):
    def variable(i, depth):
        require(type(i) is int and i != 0 and -scope <= i <= depth)
        return ['v', i]
    walk(a, variable, sort=sort)


def opening(a, value, depth=0):
    return walk(a, lambda i, d: value if i == d + 1 else ['v', i - (i > d + 1)], depth)


def derive(proof, context, scope):
    match proof:
        case ['r', a]:
            syntax(a, scope)
            return ['=', a, a]
        case ['u', j, t, p]:
            require(type(j) is int and 0 <= j < len(context))
            syntax(t, scope)
            base, body = context[j]
            require(derive(p, context, scope) == base)
            return opening(body, t)
        case ['v', p, q]:
```

```
            _, s, t = derive(p, context, scope)
            _, (_, counter, seed, body), r = derive(q, context, scope)
            if counter == ['z']:
                require(seed == s)
            else:
                require(counter[0] == 's' and s == ['i', counter[1], seed, body])
                t = opening(body, t)
            return ['=', r, t]
        case ['e', p, q]:
            paired = derive(p, context, scope)
            require(paired[0] == 'd')
            body = opening(paired[1], ['v', -scope - 1], 1)
            clause = (opening(body, ['z']), body)
            a = derive(q, context + [clause], scope + 1)
            syntax(a, scope, 1)
            return a
        case ['i', b, t, p]:
            syntax(['d', b], scope, 1)
            syntax(t, scope)
            fresh = ['v', -scope - 1]
            body = opening(b, t, 1)
            eta = fresh if b[0] == '=' and t == ['z'] else ['z']
            clause = (['=', ['z'], ['z']], opening(body, eta))
            require(derive(p, context + [clause], scope + 1) == opening(body, ['s', fresh]))
            return ['d', b]
        case _:
            raise ValueError('invalid proof')


def verify(data):
    require(type(data) is dict and set(data) == {'goal', 'proof'})
    syntax(data['goal'], 0, 1)
    require(derive(data['proof'], [], 0) == data['goal'])


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```